

Hello World для MK61s mini: пошагово

Версия документа: 14.09.2026. APP ABI 4.

Ниже только практические действия: создать один файл, собрать HELLO.APP и запустить его на калькуляторе.

Перед началом

- исходники проекта MK61S_MINI;
- Python 3;
- Arduino IDE с установленным пакетом плат STM32.

Откройте Терминал в macOS/Linux или PowerShell в Windows и перейдите в корень проекта - каталог, где находятся папки tools, sdk и examples.

Шаг 1. Создайте исходник

Создайте каталог my-app, а в нём файл main.c. Вставьте этот код целиком:

```
#include "mk61_app.h"

int main(void) {
    const uint32_t needs =
        MK61_APP_CAP_TEXT_DISPLAY | MK61_APP_CAP_KEYBOARD;

    if(!mk61_app_api_compatible(mk61_api, sizeof(*mk61_api), needs))
        return MK61_APP_RUNTIME_ERROR;

    static const char text[] = "HELLO, MK61S!";

    if(!mk61_api->display_clear())
        return MK61_APP_RUNTIME_ERROR;

    if(!mk61_api->display_write_utf8(0, 0, text, sizeof(text) - 1))
        return MK61_APP_RUNTIME_ERROR;

    (void) mk61_api->key_wait();
    return MK61_APP_OK;
}
```

Сохраните файл как:

MK61S_MINI/my-app/main.c

Шаг 2. Соберите APP

Выберите свою ОС и вставьте весь блок команд. Он сам найдёт установленную версию ARM GCC из пакета STM32. Arduino IDE запускать не нужно.

macOS

```
ARM="$HOME/Library/Arduino15/packages/STMicroelectronics"
ARM_BIN="$(printf '%s\n' \
    "$ARM/tools/xbpack-arm-none-eabi-gcc"/*bin | head -n 1)"
python3 tools/build_portable_app.py --name HELLO \
    --source my-app/main.c --arm-toolchain-bin "$ARM_BIN" \
    --output-dir .build/hello
```

Linux

```
ARM="$HOME/.arduino15/packages/STMicroelectronics"
ARM_BIN="$(printf '%s\n' \
    "$ARM/tools/xbpack-arm-none-eabi-gcc"/*bin | head -n 1)"
python3 tools/build_portable_app.py --name HELLO \
    --source my-app/main.c --arm-toolchain-bin "$ARM_BIN" \
    --output-dir .build/hello
```

Windows PowerShell

```
$Arm = "$env:LOCALAPPDATA\Arduino15\packages\STMicroelectronics"
$ArmBin = (Get-Item "$Arm\tools\pack-arm-none-eabi-gcc\*\bin" |
  Select-Object -First 1).FullName
py -3 tools\build_portable_app.py --name HELLO `
  --source my-app\main.c --arm-toolchain-bin "$ArmBin" `
  --output-dir .build\hello
```

Если Windows не знает команду `py -3`, замените её на `python`. После успешной сборки файл находится в `.build/hello/HELLO.APP`.

На калькулятор нужен только этот файл. `HELLO.elf`, `HELLO.bin`, `HELLO.map` и `HELLO.json` копировать не нужно.

Шаг 3. Скопируйте APP на MK61s

1. На калькуляторе откройте Меню -> USB-диск.
2. На компьютере откройте диск MK61S C5.
3. Создайте на нём каталог Apps, если его ещё нет.
4. Скопируйте `.build/hello/HELLO.APP` в каталог Apps.
5. Безопасно извлеките диск на компьютере.
6. Нажмите ESC на калькуляторе, чтобы выйти из режима USB-диска.
7. В Проводнике калькулятора откройте Apps/HELLO.APP клавишей OK.

На экране появится строка `HELLO, MK61S!`. Нажмите любую клавишу, чтобы закрыть приложение.

Как изменить надпись

Текст находится в этой строке:

```
static const char text[] = "HELLO, MK61S!";
```

Замените текст между кавычками, сохраните `main.c` и снова выполните ту же команду сборки. Новый `.build/hello/HELLO.APP` заменит предыдущий.

Если не собирается

- `tools/build_portable_app.py` не найден: терминал открыт не в корне проекта.
- `ARM GCC not found`: в Arduino IDE не установлен пакет плат STM32.
- APP не запускается: обновите прошивку и комплект /System до версии с пользовательскими APP ABI 4.

Для Hello World больше ничего настраивать не требуется. Полное руководство по файлам, графике, звуку и памяти находится в [MK61s-mini-APP-Programming](#).

Тот же Hello World на Rust

Результат тот же: APP очищает экран, показывает HELLO, MK61S! и ждёт нажатия клавиши. Дополнительно должен быть установлен Rust через rustup.

Шаг 1. Установите цель для MK61s

Одинаковая команда для macOS, Linux и Windows PowerShell:

```
rustup target add thumbv7em-none-eabihf
```

Шаг 2. Создайте исходник

Создайте каталог my-app-rust, а в нём файл main.rs с этим кодом:

```
#![no_std]

#[path = "../sdk/portable/rust/mk61_app.rs"]
mod mk61;

use core::panic::PanicInfo;

#[no_mangle]
pub extern "C" fn main() -> i32 {
    let needs = mk61::CAP_TEXT_DISPLAY | mk61::CAP_KEYBOARD;
    if !mk61::api_compatible(needs)
        || !mk61::display_clear()
        || !mk61::display_write_utf8(0, 0, b"HELLO, MK61S!")
    {
        return mk61::APP_RUNTIME_ERROR;
    }
    let _ = mk61::key_wait();
    mk61::APP_OK
}

#[panic_handler]
fn panic(_: &PanicInfo) -> ! {
    loop {}
}
```

Шаг 3. Соберите и скопируйте APP

Сначала задайте ARM_BIN или \$ArmBin первыми строками блока для своей ОС из шага 2 на предыдущих страницах. Затем выполните команду сборки.

macOS и Linux:

```
python3 tools/build_portable_app.py --name HELLO_RS \
  --source my-app-rust/main.rs --arm-toolchain-bin "$ARM_BIN" \
  --output-dir .build/hello-rust
```

Windows PowerShell:

```
py -3 tools\build_portable_app.py --name HELLO_RS `
  --source my-app-rust\main.rs --arm-toolchain-bin "$ArmBin" `
  --output-dir .build\hello-rust
```

Скопируйте .build/hello-rust/HELLO_RS.APP на калькулятор точно так же, как HELLO.APP в шаге 3. Остальные файлы из каталога сборки не нужны.